

PROYECTO FINAL

BASES DE DATOS

Sistema de Gestión de Biblioteca Pública
Escenario C

Documento Técnico

Motor de base de datos: PostgreSQL 17
Notación del modelo conceptual: Entidad-Relación (Chen)

Integrantes del equipo:

1. Canjura Paz, Diego Fernando | 00042025
2. Erazo Durán, Jorge Alejandro | 00047225
3. Olmedo Lobos, César Andrés | 00033225
4. Peraza Navas, William Eduardo | 00087925
5. Pineda Cáceres, Flavio César | 00115625
6. Serrano Cardona, René Fernando | 00098725

Antiguo Cuscatlán, El Salvador, junio del 2026

Tabla de contenido

1. Descripción del escenario y reglas de negocio.....	1
1.1 Contexto.....	1
1.2 Entidades principales del sistema.....	1
1.3 Reglas de negocio.....	1
1.4 Identificación del usuario	2
2. Diagrama Entidad–Relación (notación Chen).....	3
2.1 Resumen de entidades y atributos clave	3
2.2 Relaciones principales.....	4
3. Esquema relacional normalizado (3FN)	5
3.1 Verificación de formas normales.....	5
3.2 Listado de tablas	5
4. Diccionario de datos.....	7
5. Documentación de consultas frecuentes.....	11
5.1 Libros más prestados en los últimos 3 meses	11
5.2 Socios con multas pendientes	11
5.3 Autores con mayor número de títulos en catálogo	11
5.4 Ejemplares que nunca han sido prestados	12
5.5 Empleado que ha procesado más préstamos en el mes.....	12
5.6 Reporte complementario A: préstamos pendientes de devolución.....	12
5.7 Reporte complementario B: estado actual de los ejemplares.....	13
6. Documentación de funciones, procedimientos y disparadores	14
6.1 Función: calcular_monto_multa	14
6.2 Trigger: tr_generar_multa	14
6.3 Procedimiento almacenado: sp_registrar_prestamo	16
7. Conclusiones	19

1. Descripción del escenario y reglas de negocio

1.1 Contexto

Una biblioteca pública requiere un sistema de información para gestionar de forma integral su catálogo bibliográfico, los socios registrados, el ciclo de préstamos y devoluciones, la generación automática de multas por retraso, y las reservas de material bibliográfico cuando los ejemplares se encuentran en circulación.

El sistema debe centralizar la información que actualmente podría manejarse de forma manual o dispersa, garantizando la integridad de los datos mediante restricciones a nivel de base de datos y automatizando procesos de negocio críticos como el cálculo de multas y la validación de elegibilidad para nuevos préstamos.

1.2 Entidades principales del sistema

El modelo de datos se compone de once entidades principales, descritas a continuación:

- Libro: representa un título bibliográfico identificado por su ISBN.
- Autor: persona que escribe uno o más libros del catálogo.
- Editorial: casa editorial responsable de la publicación de un libro.
- Categoría: clasificación temática asignada a cada libro.
- Ejemplar: copia física de un libro, con su propio estado de disponibilidad y ubicación.
- Socio: persona registrada en la biblioteca con derecho a solicitar préstamos.
- Empleado: personal de la biblioteca que procesa préstamos y devoluciones.
- Prestamo: registro de la entrega de un ejemplar a un socio.
- Devolución: registro del regreso de un ejemplar prestado.
- Multa: sanción económica generada automáticamente cuando una devolución se realiza con retraso.
- Reserva: solicitud de un socio para apartar un título cuando todos sus ejemplares están en préstamo.

1.3 Reglas de negocio

Las siguientes reglas de negocio gobiernan el comportamiento del sistema y fueron implementadas mediante restricciones de integridad, triggers y procedimientos almacenados:

1. Un libro puede tener múltiples autores, y un autor puede haber escrito varios libros (relación N:M).
2. Un libro puede tener varios ejemplares físicos disponibles en el catálogo.
3. Un socio puede tener hasta 3 préstamos activos de forma simultánea. Esta validación es aplicada por el procedimiento almacenado `sp_registrar_prestamo`.
4. Si una devolución se realiza después de la fecha límite establecida, el sistema genera automáticamente una multa calculada en función de los días de retraso, mediante el trigger `tr_generar_multa`.

5. Un socio con multas en estado Pendiente, o con algún préstamo activo cuya fecha límite ya venció sin haberse registrado la devolución, no puede realizar nuevos préstamos hasta regularizar su situación.
6. Los ejemplares pueden ser reservados por un socio cuando el título se encuentra completamente en circulación.
7. Cada préstamo es procesado por un empleado en el momento de la salida, y cada devolución es procesada por un empleado (posiblemente distinto) en el momento de la recepción.

1.4 Identificación del usuario

El sistema cuenta con dos tipos de actores principales: el empleado de la biblioteca, encargado de operar el sistema desde el mostrador de atención, y el socio, cuya información se gestiona dentro del sistema, aunque no interactúe directamente con la base de datos.

2. Diagrama Entidad–Relación (notación Chen)

El siguiente diagrama representa el modelo conceptual de datos en notación Chen, en el que las entidades se representan mediante rectángulos, los atributos mediante óvalos, y las relaciones mediante rombos. Las cardinalidades (1, N, M) se indican junto a cada línea de conexión.

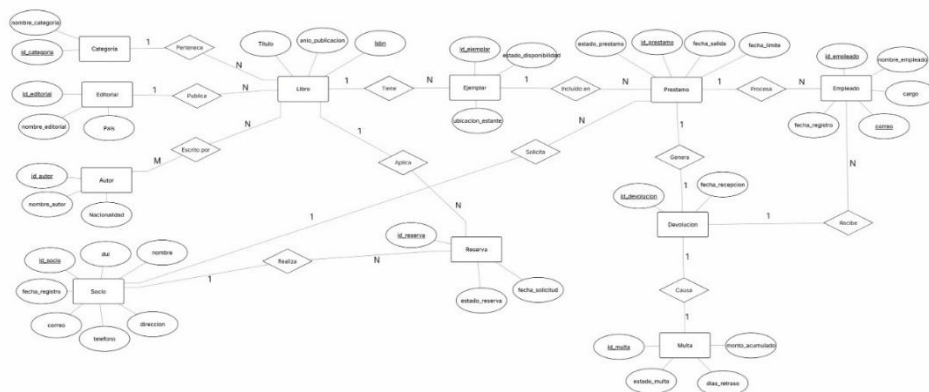


Figura 1. Diagrama Entidad-Relación en notación Chen del sistema de biblioteca

2.1 Resumen de entidades y atributos clave

Entidad	Atributos principales	Identificador
Libro	isbn, titulo, anio_publicacion	isbn
Autor	id_autor, nombre_autor, nacionalidad	id_autor
Editorial	id_editorial, nombre_editorial, pais	id_editorial
Categoría	id_categoria, nombre_categoria	id_categoria
Ejemplar	id_ejemplar, isbn, estado_disponibilidad, ubicacion_estante	id_ejemplar
Socio	id_socio, dui, nombre, correo, telefono	id_socio
Empleado	id_empleado, nombre_empleado, cargo, correo	id_empleado
Prestamo	id_prestamo, estado_prestamo, fecha_salida, fecha_limite	id_prestamo
Devolucion	id_devolucion, fecha_recepcion	id_devolucion
Multa	id_multa, dias_retraso, monto_acumulado, estado_multa	id_multa
Reserva	id_reserva, fecha_solicitud, estado_reserva	id_reserva

2.2 Relaciones principales

Relación	Entidades	Cardinalidad
escribe	Autor — Libro	N : M
pertenece	Libro — Categoria	N : 1
publicada_por	Libro — Editorial	N : 1
tiene	Libro — Ejemplar	1 : N
realiza	Socio — Prestamo	1 : N
procesa_salida	Empleado — Prestamo	1 : N
genera	Prestamo — Devolucion	1 : 1
recibe	Empleado — Devolucion	1 : N
origina	Devolucion — Multa	1 : 0..1
hace_reserva	Socio — Reserva	1 : N
reservado_en	Libro — Reserva	1 : N

3. Esquema relacional normalizado (3FN)

A partir del modelo conceptual, se aplicó el proceso de transformación a un esquema relacional, resolviendo la relación N:M mediante una tabla intermedia y verificando el cumplimiento de la Primera, Segunda y Tercera Forma Normal en cada una de las doce tablas resultantes.

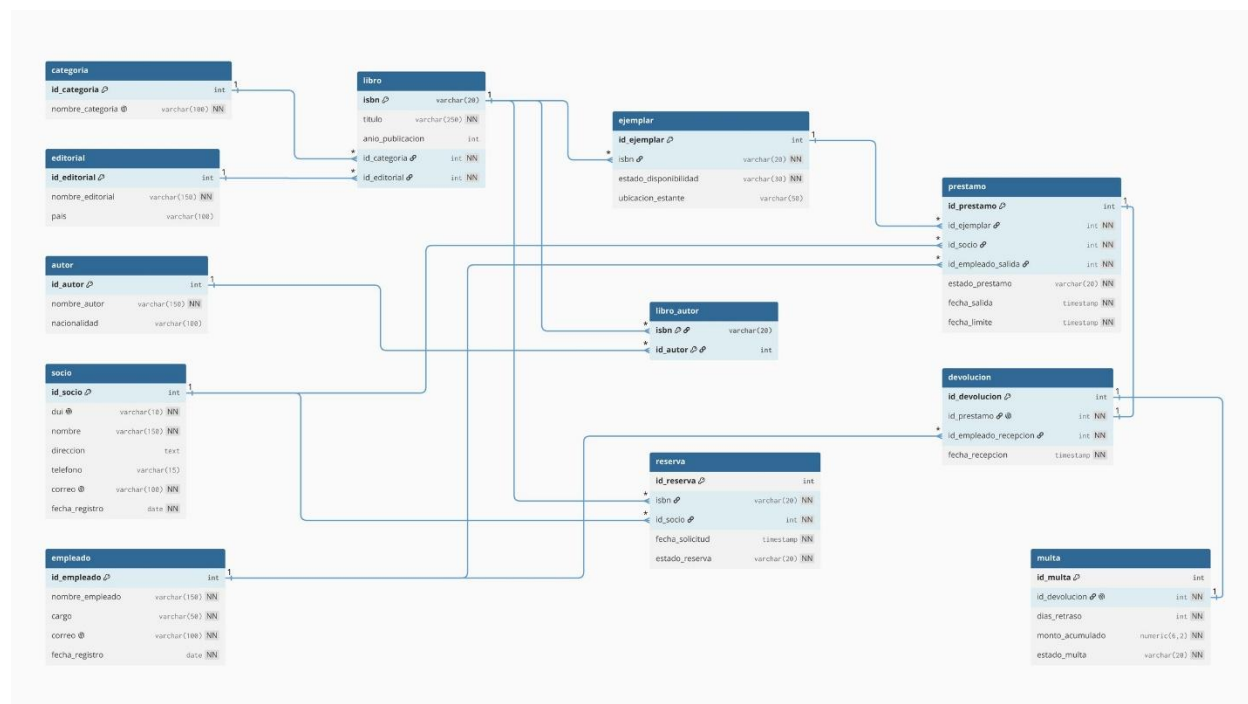


Figura 2. Esquema relacional normalizado en Tercera Forma Normal

3.1 Verificación de formas normales

- **1FN**: todos los atributos de cada tabla son atómicos. No existen listas ni grupos repetidos; por ejemplo, los autores de un libro no se almacenan en un solo campo, sino en la tabla intermedia libro_autor.
- **2FN**: en la única tabla con clave primaria compuesta (libro_autor), no existen dependencias parciales; el resto de las tablas usan clave simple, por lo que la 2FN se cumple automáticamente.
- **3FN**: no existen dependencias transitivas. Por ejemplo, el nombre de la editorial no se almacena en la tabla libro, sino en su propia tabla editorial, referenciada mediante clave foránea.

3.2 Listado de tablas

Tabla	Clave primaria	Claves foráneas
categoria	id_categoria	—

editorial	id_editorial	—
autor	id_autor	—
libro	isbn	id_categoria → categoria, id_editorial → editorial
libro_autor	isbn + id_autor	isbn → libro, id_autor → autor
ejemplar	id_ejemplar	isbn → libro
socio	id_socio	—
empleado	id_empleado	—
prestamo	id_prestamo	id_ejemplar, id_socio, id_empleado_salida
devolucion	id_devolucion	id_prestamo, id_empleado_recepcion
multa	id_multa	id_devolucion
reserva	id_reserva	isbn → libro, id_socio → socio

4. Diccionario de datos

A continuación, se detalla, campo por campo, la estructura completa de las doce tablas que conforman la base de datos biblioteca_db, según fueron implementadas en el script DDL.

categoria

Campo	Tipo	Restricciones	Descripción
id_categoria	INT (Identity)	PRIMARY KEY	Identificador autogenerado de la categoría.
nombre_categoria	VARCHAR(100)	UNIQUE, NOT NULL	Nombre de la clasificación temática.

editorial

Campo	Tipo	Restricciones	Descripción
id_editorial	INT (Identity)	PRIMARY KEY	Identificador autogenerado de la editorial.
nombre_editorial	VARCHAR(150)	NOT NULL	Nombre de la casa editorial.
pais	VARCHAR(100)	—	País de origen de la editorial.

autor

Campo	Tipo	Restricciones	Descripción
id_autor	INT (Identity)	PRIMARY KEY	Identificador autogenerado del autor.
nombre_autor	VARCHAR(150)	NOT NULL	Nombre completo del autor.
nacionalidad	VARCHAR(100)	—	Nacionalidad del autor.

libro

Campo	Tipo	Restricciones	Descripción
isbn	VARCHAR(20)	PRIMARY KEY	Identificador internacional del libro (clave natural).
titulo	VARCHAR(250)	NOT NULL	Título del libro.
anio_publicacion	INT	—	Año en que fue publicado.
id_categoria	INT	FK → categoria	Categoría temática del libro.
id_editorial	INT	FK → editorial	Editorial responsable de la publicación.

libro_autor

Campo	Tipo	Restricciones	Descripción
isbn	VARCHAR(20)	PK (compuesta), FK → libro, ON DELETE CASCADE	Referencia al libro.
id_autor	INT	PK (compuesta), FK → autor, ON DELETE CASCADE	Referencia al autor.

ejemplar

Campo	Tipo	Restricciones	Descripción
id_ejemplar	INT (Identity)	PRIMARY KEY	Identificador autogenerado del ejemplar físico.
isbn	VARCHAR(20)	FK → libro	Libro al que pertenece este ejemplar.
estado_disponibilidad	VARCHAR(30)	NOT NULL, DEFAULT 'Disponible', CHECK IN (4 valores)	Estado actual: Disponible, Prestado, Reservado o Mantenimiento.
ubicacion_estante	VARCHAR(50)	—	Ubicación física dentro de la biblioteca.

socio

Campo	Tipo	Restricciones	Descripción
id_socio	INT (Identity)	PRIMARY KEY	Identificador autogenerado del socio.
dui	VARCHAR(10)	UNIQUE, NOT NULL, CHECK formato 8 dígitos-1 dígito	Documento Único de Identidad del socio.
nombre	VARCHAR(150)	NOT NULL	Nombre completo del socio.
direccion	TEXT	—	Dirección de residencia.
telefono	VARCHAR(15)	UNIQUE, CHECK formato 6/7 + 3 dígitos-4 dígitos	Número de teléfono de contacto.
correo	VARCHAR(100)	UNIQUE, NOT NULL, CHECK formato correo válido	Correo electrónico del socio.
fecha_registro	DATE	NOT NULL, DEFAULT CURRENT_DATE	Fecha de inscripción a la biblioteca.

empleado

Campo	Tipo	Restricciones	Descripción
id_empleado	INT (Identity)	PRIMARY KEY	Identificador autogenerado del empleado.
nombre_empleado	VARCHAR(150)	NOT NULL	Nombre completo del empleado.
cargo	VARCHAR(50)	NOT NULL	Puesto o cargo dentro de la biblioteca.
correo	VARCHAR(100)	UNIQUE, NOT NULL, CHECK formato correo válido	Correo institucional del empleado.
fecha_registro	DATE	NOT NULL, DEFAULT CURRENT_DATE	Fecha de contratación o registro.

préstamo

Campo	Tipo	Restricciones	Descripción
id_prestamo	INT (Identity)	PRIMARY KEY	Identificador autogenerado del préstamo.
id_ejemplar	INT	FK → ejemplar	Ejemplar físico entregado.
id_socio	INT	FK → socio	Socio que recibe el préstamo.
id_empleado_salida	INT	FK → empleado	Empleado que procesa la salida.
estado_prestamo	VARCHAR(20)	NOT NULL, DEFAULT 'Activo', CHECK IN (3 valores)	Estado: Activo, Devuelto o Vencido.
fecha_salida	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora de entrega del ejemplar.
fecha_limite	TIMESTAMP	NOT NULL, CHECK > fecha_salida	Fecha límite para la devolución.

devolución

Campo	Tipo	Restricciones	Descripción
id_devolucion	INT (Identity)	PRIMARY KEY	Identificador autogenerado de la devolución.
id_prestamo	INT	UNIQUE, NOT NULL, FK → prestamo	Préstamo que se está devolviendo (relación 1:1).
id_empleado_recepcion	INT	FK → empleado	Empleado que recibe el ejemplar.
fecha_recepcion	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora real de la devolución.

multa

Campo	Tipo	Restricciones	Descripción
id_multa	INT (Identity)	PRIMARY KEY	Identificador autogenerado de la multa.
id_devolucion	INT	UNIQUE, NOT NULL, FK → devolucion	Devolución que originó la multa (relación 1:1).
dias_retraso	INT	NOT NULL, CHECK > 0	Número de días que excedió la fecha límite.
monto_acumulado	NUMERIC(6,2)	NOT NULL, CHECK > 0	Monto total a pagar (dias_retraso × \$0.25).
estado_multa	VARCHAR(20)	NOT NULL, DEFAULT 'Pendiente', CHECK IN (2 valores)	Estado: Pendiente o Pagada.

reserva

Campo	Tipo	Restricciones	Descripción
id_reserva	INT (Identity)	PRIMARY KEY	Identificador autogenerado de la reserva.
isbn	VARCHAR(20)	FK → libro	Título solicitado en reserva.
id_socio	INT	FK → socio	Socio que realiza la reserva.
fecha_solicitud	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora en que se solicitó la reserva.
estado_reserva	VARCHAR(20)	NOT NULL, DEFAULT 'Activa', CHECK IN (3 valores)	Estado: Activa, Completada o Cancelada.

5. Documentación de consultas frecuentes

Esta sección documenta las cinco consultas SQL desarrolladas para responder a las necesidades del negocio planteadas en el enunciado del proyecto, junto con dos reportes complementarios de apoyo operativo. El código completo se encuentra en el archivo consultas.sql.

5.1 Libros más prestados en los últimos 3 meses

Identifica los títulos con mayor demanda en el trimestre reciente, uniendo prestamo con ejemplar y libro, filtrando por fecha_salida y agrupando por isbn.

```
SELECT l.isbn, l.titulo, COUNT(p.id_prestamo) AS total_prestamos
FROM prestamo p
JOIN ejemplar e ON e.id_ejemplar = p.id_ejemplar
JOIN libro l ON l.isbn = e.isbn
WHERE p.fecha_salida >= CURRENT_TIMESTAMP - INTERVAL '3 months'
GROUP BY l.isbn, l.titulo
ORDER BY total_prestamos DESC;
```

5.2 Socios con multas pendientes

Lista los socios con multas en estado Pendiente y también a quienes, sin tener aún una multa formalizada, mantienen un préstamo activo cuya fecha límite ya venció. Esto permite identificar con antelación a los socios que serán bloqueados por el procedimiento sp_registrar_prestamo en cuanto se procese su devolución.

```
SELECT s.id_socio, s.nombre, s.correo, s.telefono,
       COUNT(DISTINCT m.id_multa) AS cantidad_multas,
       SUM(m.monto_acumulado) AS deuda_total
FROM socio s
JOIN prestamo p ON p.id_socio = s.id_socio
LEFT JOIN devolucion d ON d.id_prestamo = p.id_prestamo
LEFT JOIN multa m ON m.id_devolucion = d.id_devolucion
WHERE m.estado_multa = 'Pendiente'
       OR (p.estado_prestamo = 'Activo' AND p.fecha_limite < CURRENT_DATE)
GROUP BY s.id_socio, s.nombre, s.correo, s.telefono
ORDER BY deuda_total DESC;
```

5.3 Autores con mayor número de títulos en catálogo

Cuenta los títulos distintos asociados a cada autor mediante la tabla intermedia libro_autor.

```
SELECT a.id_autor, a.nombre_autor, a.nacionalidad,
       COUNT(la.isbn) AS total_titulos
FROM autor a
JOIN libro_autor la ON la.id_autor = a.id_autor
GROUP BY a.id_autor, a.nombre_autor, a.nacionalidad
ORDER BY total_titulos DESC;
```

5.4 Ejemplares que nunca han sido prestados

Detecta material físico sin movimiento, filtrando los ejemplares cuyo id_ejemplar no aparece en ningún registro de préstamo.

```
SELECT e.id_ejemplar, e.ubicacion_estante,
       e.estado_disponibilidad, l.isbn, l.titulo
FROM ejemplar e
JOIN libro l ON l.isbn = e.isbn
WHERE e.id_ejemplar NOT IN (
    SELECT DISTINCT id_ejemplar FROM prestamo
)
ORDER BY l.titulo, e.id_ejemplar;
```

5.5 Empleado que ha procesado más préstamos en el mes

Identifica al empleado con mayor número de préstamos gestionados durante el mes en curso. La consulta agrupa por empleado y, mediante LIMIT 1, devuelve únicamente al de mayor desempeño en el período.

```
SELECT em.id_empleado, em.nombre_empleado, em.cargo,
       COUNT(p.id_prestamo) AS prestamos_procesados
FROM empleado em
JOIN prestamo p ON p.id_empleado_salida = em.id_empleado
WHERE DATE_TRUNC('month', p.fecha_salida)
      = DATE_TRUNC('month', CURRENT_DATE)
GROUP BY em.id_empleado, em.nombre_empleado, em.cargo
ORDER BY prestamos_procesados DESC
LIMIT 1;
```

5.6 Reporte complementario A: préstamos pendientes de devolución

Muestra todos los préstamos que todavía no tienen una devolución registrada, indicando si la fecha límite ya fue superada. Es el reporte de control diario que el personal de mostrador consulta para dar seguimiento a los préstamos en curso.

```
SELECT p.id_prestamo, s.nombre AS socio, l.titulo AS libro,
       p.fecha_salida, p.fecha_limite,
       CASE
           WHEN p.fecha_limite < CURRENT_DATE THEN 'Vencido'
           ELSE 'En tiempo'
       END AS estado_entrega
FROM prestamo p
JOIN socio s ON p.id_socio = s.id_socio
JOIN ejemplar e ON p.id_ejemplar = e.id_ejemplar
JOIN libro l ON e.isbn = l.isbn
WHERE p.id_prestamo NOT IN (
    SELECT id_prestamo FROM devolucion
)
ORDER BY p.fecha_limite;
```

5.7 Reporte complementario B: estado actual de los ejemplares

Presenta un inventario completo de todos los ejemplares registrados, junto con su estado de disponibilidad y ubicación física dentro de la biblioteca. Sirve como apoyo para tareas de inventario y reorganización del estante.

```
SELECT l.titulo, e.id_ejemplar,  
       e.estado_disponibilidad, e.ubicacion_estante  
FROM ejemplar e  
JOIN libro l ON e.isbn = l.isbn  
ORDER BY l.titulo, e.id_ejemplar;
```

6. Documentación de funciones, procedimientos y disparadores

El comportamiento procedimental de la base de datos se implementó en el archivo programacion.sql, mediante una función auxiliar, un trigger automático y un procedimiento almacenado con validaciones de negocio.

6.1 Función: calcular_monto_multa

Función auxiliar que recibe el número de días de retraso y devuelve el monto a cobrar, aplicando una tarifa fija de \$0.25 por día. Es invocada internamente por el trigger tr_generar_multa.

Parámetro	Tipo	Descripción
p_dias_retraso	INT	Número de días que excedió la fecha límite del préstamo.

Retorna: NUMERIC(6,2) — monto total calculado.

```
CREATE OR REPLACE FUNCTION calcular_monto_multa(p_dias_retraso INT)
RETURNS NUMERIC(6,2) LANGUAGE plpgsql AS $$
DECLARE
    v_tarifa_diaria NUMERIC(6,2) := 0.25;
BEGIN
    IF p_dias_retraso <= 0 THEN
        RETURN 0.00;
    END IF;
    RETURN p_dias_retraso * v_tarifa_diaria;
END;
$$;
```

6.2 Trigger: tr_generar_multa

Se activa automáticamente después de cada INSERT en la tabla devolucion. Calcula los días de retraso comparando la fecha de recepción con la fecha límite del préstamo asociado; si hay retraso, inserta el registro de multa correspondiente, y en todos los casos actualiza el estado del préstamo a Devuelto y libera el ejemplar a estado Disponible. El cálculo del retraso se realiza en segundos mediante EXTRACT(EPOCH...) y se redondea hacia arriba con CEIL, de modo que cualquier fracción de día de retraso, por mínima que sea, se contabiliza como un día completo. Si por alguna razón no se encuentra el préstamo asociado a la devolución, el trigger detiene la operación y notifica el error en lugar de continuar de forma silenciosa.

Propiedad	Valor
Evento de activación	AFTER INSERT ON devolucion
Frecuencia	FOR EACH ROW
Tablas afectadas	multa (INSERT condicional), prestamo (UPDATE), ejemplar (UPDATE)
Manejo de excepciones	NO_DATA_FOUND si el préstamo asociado no existe


```
CREATE OR REPLACE FUNCTION fn_generar_multa()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_fecha_limite TIMESTAMP;
    v_dias_retraso INT := 0;
    v_monto NUMERIC(6,2) := 0.00;
    v_id_ejemplar INT;
BEGIN
    SELECT fecha_limite, id_ejemplar
    INTO STRICT v_fecha_limite, v_id_ejemplar
    FROM prestamo WHERE id_prestamo = NEW.id_prestamo;

    IF NEW.fecha_recepcion > v_fecha_limite THEN
        v_dias_retraso := CEIL(
            EXTRACT(EPOCH FROM (NEW.fecha_recepcion - v_fecha_limite))
            / 86400
        )::INT;
    END IF;

    IF v_dias_retraso > 0 THEN
        v_monto := calcular_monto_multa(v_dias_retraso);
        INSERT INTO multa (id_devolucion, dias_retraso,
                           monto_acumulado, estado_multa)
        VALUES (NEW.id_devolucion, v_dias_retraso, v_monto, 'Pendiente');
    END IF;

    UPDATE prestamo SET estado_prestamo = 'Devuelto'
    WHERE id_prestamo = NEW.id_prestamo;

    UPDATE ejemplar SET estado_disponibilidad = 'Disponible'
    WHERE id_ejemplar = v_id_ejemplar;

    RETURN NEW;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'No se encontró el préstamo asociado.';
END;
$$;

CREATE TRIGGER tr_generar_multa
AFTER INSERT ON devolucion
FOR EACH ROW
EXECUTE FUNCTION fn_generar_multa();
```

6.3 Procedimiento almacenado: sp_registrar_prestamo

Procesa el préstamo de un ejemplar realizando tres validaciones de negocio en orden estricto antes de registrar la operación: disponibilidad del ejemplar, situación crediticia del socio, y verificación de que no se supere el límite de 3 préstamos activos.

Parámetro	Tipo	Descripción
p_id_ejemplar	INT	Ejemplar físico que se desea prestar.
p_id_socio	INT	Socio que solicita el préstamo.
p_id_empleado_salida	INT	Empleado que procesa la operación.
p_dias_prestamo	INT	Duración del préstamo, en días.

Validaciones aplicadas, en orden de ejecución:

8. El ejemplar debe existir y su estado_disponibilidad debe ser 'Disponible'.
9. El socio no debe tener ninguna multa con estado_multa = 'Pendiente', ni ningún préstamo con estado_prestamo = 'Activo' cuya fecha_limite ya haya vencido. Ambos criterios se combinan mediante UNION en una sola verificación.
10. El socio no debe tener 3 o más préstamos con estado_prestamo = 'Activo'.

Si alguna validación falla, el procedimiento lanza una excepción mediante RAISE EXCEPTION con un mensaje descriptivo, y no se realiza ninguna inserción.

```
CREATE OR REPLACE PROCEDURE sp_registrar_prestamo(
    p_id_ejemplar INT, p_id_socio INT,
    p_id_empleado_salida INT, p_dias_prestamo INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_estado_ejemplar VARCHAR(30);
    v_multas_pendientes INT;
    v_prestamos_activos INT;
    v_fecha_limite TIMESTAMP;
BEGIN
    -- Validación 1: ejemplar disponible
    SELECT estado_disponibilidad INTO v_estado_ejemplar
    FROM ejemplar WHERE id_ejemplar = p_id_ejemplar;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'El ejemplar % no existe.', p_id_ejemplar;
    END IF;
    IF v_estado_ejemplar <> 'Disponible' THEN
        RAISE EXCEPTION 'El ejemplar % no está disponible.', p_id_ejemplar;
    END IF;

    -- Validación 2: multas pendientes o préstamos vencidos sin devolver
    SELECT COUNT(*) INTO v_multas_pendientes
    FROM (
        SELECT m.id_multa
        FROM prestamo p
        JOIN devolucion d ON d.id_prestamo = p.id_prestamo
        JOIN multa m ON m.id_devolucion = d.id_devolucion
        WHERE p.id_socio = p_id_socio AND m.estado_multa = 'Pendiente'
        UNION
        SELECT id_prestamo
        FROM prestamo
        WHERE id_socio = p_id_socio AND estado_prestamo = 'Activo'
        AND fecha_limite < CURRENT_TIMESTAMP
    ) AS multas_y_retrasos;
    IF v_multas_pendientes > 0 THEN
        RAISE EXCEPTION 'El socio tiene multas pendientes o préstamos vencidos.';
    END IF;

    -- Validación 3: límite de préstamos activos
    SELECT COUNT(*) INTO v_prestamos_activos
    FROM prestamo
    WHERE id_socio = p_id_socio AND estado_prestamo = 'Activo';
    IF v_prestamos_activos >= 3 THEN
        RAISE EXCEPTION 'El socio % alcanzó el límite de préstamos.', p_id_socio;
    END IF;

    -- Registro del préstamo
    v_fecha_limite := CURRENT_TIMESTAMP + (p_dias_prestamo || ' days')::INTERVAL;
    INSERT INTO prestamo (id_ejemplar, id_socio, id_empleado_salida,
        estado_prestamo, fecha_salida, fecha_limite)
    VALUES (p_id_ejemplar, p_id_socio, p_id_empleado_salida,
```

```
        'Activo', CURRENT_TIMESTAMP, v_fecha_limite);

    UPDATE ejemplar SET estado_disponibilidad = 'Prestado'
    WHERE id_ejemplar = p_id_ejemplar;
END;
$$;
```

Ejemplo de invocación: CALL sp_registrar_prestamo(1, 2, 1, 15);

7. Conclusiones

El desarrollo de este proyecto permitió aplicar de manera integral el ciclo de vida del diseño de bases de datos relacionales, desde la conceptualización del problema de negocio hasta su implementación funcional en PostgreSQL 17.

El proceso de normalización resultó especialmente relevante para evitar redundancias: decisiones como usar el ISBN como clave primaria natural en la tabla libro, o separar la relación reserva hacia el título en lugar del ejemplar físico, simplificaron tanto el modelo como las consultas posteriores.

La incorporación de lógica procedimental —en particular el trigger de generación automática de multas y el procedimiento de validación de préstamos— demuestra cómo las reglas de negocio críticas pueden y deben aplicarse a nivel de motor de base de datos, garantizando su cumplimiento independientemente de la aplicación cliente que consume estos datos.

Finalmente, el uso de restricciones CHECK con expresiones regulares para validar formatos de documento de identidad, teléfono y correo electrónico refuerza la integridad de los datos desde el momento mismo de su inserción, reduciendo la necesidad de validaciones redundantes en capas superiores del sistema.